

SCHEMASTORE VIEW self-reference fails on union'd exploded tables

Zetaris AI Data Gateway — SQL Workspace · Reported by Michael Hay · 2026-09-19

Summary

Any query that references the same SCHEMASTORE VIEW more than once — via an explicit self-join, a WHERE ... IN (SELECT ...) subquery, or a scalar HAVING count = (SELECT ...) subquery — fails with MISSING_ATTRIBUTES.RESOLVED_ATTRIBUTE_APPEAR_IN_OPERATION whenever that view's underlying definition is a UNION ALL of branches built with LATERAL VIEW explode(...) against a REST-sourced table. Joining two *different* views works fine — the failure is specific to referencing the same compiled view twice, in any form, not self-joins or unions individually.

This has been confirmed live on two independent sources, hitting three different SQL forms of the same underlying problem. A reliable workaround exists (rewrite as a window function), but the error message gives no indication of the cause, and this blocks a common, unremarkable SQL pattern — “top row per group” and “does this group meet a threshold” questions — against exactly the kind of view this platform's own REST-onboarding pattern produces.

Evidence

1. Explicit self-join (Open Food Facts source, 2026-09-18)

View definition (simplified):

```
CREATE SCHEMASTORE VIEW all_products_ingredients_table AS
SELECT ... FROM nutella_ingredients_table
UNION ALL SELECT ... FROM cocacola_ingredients_table
UNION ALL SELECT ... FROM prince_ingredients_table
UNION ALL SELECT ... FROM pesto_ingredients_table;
-- each branch itself built with LATERAL VIEW explode(product.ingredients)
```

Failing query:

```
SELECT a.* FROM all_products_ingredients_table a
JOIN (
  SELECT product_name, MAX(percent_estimate) AS max_pct
  FROM all_products_ingredients_table GROUP BY product_name
) top ON top.product_name = a.product_name AND top.max_pct = a.percent_estimate;
```

MISSING_ATTRIBUTES.RESOLVED_ATTRIBUTE_APPEAR_IN_OPERATION – Resolved attribute(s) "product" missing from "code", "product", "status", "status_verbose" ... Attribute(s) with the same name appear in the operation: "product".

2. WHERE ... IN subquery (Statistics Canada WDS source, 2026-09-19)

View definition (simplified):

```
CREATE SCHEMASTORE VIEW changed_cubes_all_table AS
SELECT '2026-09-14' AS snapshot_date, cube.`productId` AS product_id, ...
FROM changed_cubes_20260914 LATERAL VIEW explode(object) AS cube
UNION ALL ... (4 more dates, same pattern)
```

Failing query:

```
SELECT product_id, snapshot_date, release_time
FROM changed_cubes_all_table
WHERE product_id IN (
  SELECT product_id FROM changed_cubes_all_table
  GROUP BY product_id HAVING COUNT(DISTINCT snapshot_date) = 5
```

```
);
```

MISSING_ATTRIBUTES.RESOLVED_ATTRIBUTE_APPEAR_IN_OPERATION – Resolved attribute(s) "object" missing from "object", "status" in operator !Generate explode(object#71304) ... Attribute(s) with the same name appear in the operation: "object".

3. Scalar HAVING subquery (same source, same session)

```
SELECT product_id, COUNT(DISTINCT snapshot_date) AS days_changed
FROM changed_cubes_all_table GROUP BY product_id
HAVING COUNT(DISTINCT snapshot_date) = (
  SELECT COUNT(DISTINCT snapshot_date) FROM changed_cubes_all_table
);
```

Failed with the identical error signature as case 2.

4. Control case — joining two DIFFERENT views works fine (PokéAPI source)

```
SELECT a.pokemon_name, a.base_experience, st.base_stat_total
FROM (SELECT ... FROM pikachu_abilities_table UNION ALL SELECT ... FROM charizard_abilities_table) a
JOIN (SELECT pokemon_name, SUM(base_stat) AS base_stat_total
      FROM all_pokemon_stats_table GROUP BY pokemon_name) st
ON st.pokemon_name = a.pokemon_name;
```

This joins a union of ability views against the separate stats view — two genuinely different views, neither self-referenced — and runs successfully. This isolates the failure to self-reference specifically, not to joins, unions, or exploded views in general.

Confirmed workaround

Rewriting each failing query as a window function, with the view referenced exactly once, resolves all three cases (confirmed live for case 1 and validated independently for cases 2/3):

```
SELECT product_name, ingredient_text, percent_estimate
FROM (
  SELECT product_name, ingredient_text, percent_estimate,
    ROW_NUMBER() OVER (PARTITION BY product_name ORDER BY percent_estimate DESC) AS rn
  FROM all_products_ingredients_table
) ranked WHERE rn = 1;
```

Root-cause hypothesis (unconfirmed — needs engineering's view)

Spark's Catalyst analyzer normally avoids attribute-id collisions in a self-join by giving one side fresh expression IDs, typically triggered when a DataFrame is explicitly aliased before joining it to itself. Our working theory: when Zetaris compiles a SCHEMASTORE VIEW into a stored catalog logical plan, and that plan is a UNION ALL of Generate(explode(...)) operators, referencing the same stored plan twice via separate SQL-text paths in one query (rather than a literal in-memory DataFrame self-join) does not trigger that usual de-duplication. The explode() operator leaves behind an internal struct attribute (here, product or object — the original array field, never itself selected) that ends up with conflicting expression IDs across the two references to the same view.

Impact

- Blocks a common, unremarkable SQL pattern — “top row per group” and “threshold count per group” questions — against exactly the kind of view this platform's own REST-onboarding workflow (CREATE LIGHTNING REST TABLE + CREATE SCHEMASTORE VIEW, unioned across multiple records) naturally produces.
- The error message gives no indication of the actual cause (a self-reference on a unioned, exploded view) — a user hitting this with no prior context has little to go on beyond “attribute conflict.”

- The workaround (window functions) is reliable and arguably better SQL, but requires knowing about this limitation in advance; it is not discoverable from the error alone.

Request

- Confirm whether this is a known Spark Catalyst limitation being inherited as-is, or something specific to how Zetaris compiles/caches a SCHEMASTORE VIEW's logical plan.
- Investigate whether SCHEMASTORE VIEW materialization can apply the same expression-ID-refresh Spark uses for genuine self-joins, so a stored view's plan gets fresh IDs each time it's pulled into a query, regardless of how many times the same query references it.
- Consider detecting this specific error pattern (a Generate/explode operator's leftover attribute colliding across two references to the same relation) and surfacing a clearer message, ideally suggesting the window-function rewrite directly.
- If this turns out to be a fundamental, hard-to-fix limitation, document it explicitly so users don't have to rediscover it independently, as we did across two separate sources.

Environment: Zetaris AI Data Gateway (cloud.zetaris.com), SQL Workspace. Sources: Open Food Facts and Statistics Canada WDS REST sources, onboarded via CREATE LIGHTNING REST TABLE + CREATE SCHEMASTORE VIEW for onboarding evaluation purposes. Full reproduction scripts available on request.